COMPUTER SCIENCE PROJECT
2023-24

Topic:

GST Billing System

Name : Prashant Kumar Thakur
Grade: 12
Section: ‘B’
Roll Number:
School Name: K.V Bengdubi,WB

INDEX

Signature of Student:

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my C.S teacher, **Jitendra kumar Yadav**, and my classmates for their invaluable support and guidance in completing this project. I am also deeply grateful to my family for their unwavering encouragement. Additionally, I appreciate the resources and references that have enhanced my Project. Thank you all for your significant contributions to the success of this project.

KENDRIYA VIDYALAYA BENGDUBI, (WB)

तत् त्वं पूषन् अपावृणु

केन्द्रीय विद्यालय संगठन

CERTIFICATE OF COMPLETION

This is to certify that this project is the bonafide work of

Master. . . .Prashant Kumar Thakur. . . .of Class…12B..

Roll No……………………….during the academic year 2023-24

The Project fulfills all the stated criteria and the student findings are his original work.

I recommend the project for evaluation.

External Examiner: ……………………… Teacher-in-Charge: ………………………

INTRODUCTION

In the contemporary business landscape, efficient and accurate billing systems are paramount to streamline financial transactions and ensure compliance with taxation *regulations*. The Goods and Services Tax (**GST**) Billing System presented herein stands as a testament to modernized and systematic billing practices.

➢ Purpose of the System

The primary aim of this GST Billing System is to automate the billing process, facilitating businesses in generating detailed and error-free bills for their customers. By integrating user-friendly interfaces and robust functionalities, the system strives to enhance the overall billing experience while ensuring adherence to prevailing GST norms.

➢ Key Features

1. **User-Friendly Interface:** The system boasts an intuitive user interface, allowing users to easily input customer details and itemized billing information.
2. **Dynamic Item Entry:** With the ability to dynamically add and manage items
3. **Real-Time Calculation:** The system calculates GST amounts and total billing values in real-time, minimizing errors and promoting financial accuracy.
4. **Bill Generation and Storage:** Unique bill numbers are generated for each transaction, and bills are systematically stored, enabling easy retrieval and reference for both businesses and customers.
5. **Search and Retrieval:** A robust search functionality enables users to efficiently locate and retrieve specific bills based on unique identifiers such as bill numbers.

➢ Significance

In the context of businesses grappling with complex billing processes and regulatory compliance, the implementation of this GST Billing System is pivotal. By automating billing procedures and providing a comprehensive record-keeping mechanism, the system not only reduces the margin of error but also contributes to the overall efficiency and transparency of business operations.

As we delve deeper into the intricacies of this GST Billing System, we uncover a tool designed not merely for the facilitation of billing but as an indispensable asset for businesses striving for financial accuracy, compliance, and customer satisfaction.

SOURCE CODE

```python
import tkinter as tk
from tkinter import *
from tkinter import messagebox
import os
from datetime import datetime


# Function to calculate GST amount and total amount
def calculate_gst(amount, gst_rate, qty):
    gst_amount = (amount + (amount * gst_rate / 100)) * qty
    total_amount = gst_amount
    return total_amount


# Function to generate the bill
def generate_bill():
    if customer_name_entry.get() == "" or customer_name_entry.get() == "":
        messagebox.showerror("Error", "Customer details are must")
    elif (
        not (phone_number_entry.get().isnumeric())
        or len(phone_number_entry.get()) != 10
    ):
        messagebox.showerror("Invalid Details", "Enter a Valid Phone Number")
    else:
        customer_name = (
            customer_name_entry.get()
        )  # Get customer name from the entry field
        phone_number = phone_number_entry.get()  # Get phone number from the entry field

        items = []  # List to store item details
        total_amount = 0  # Variable to keep track of the total amount

        # Loop through all item entry fields
        for i in range(len(item_entries)):
            item = item_entries[i].get()  # Get the item name from the entry field
            price = float(
                price_entries[i].get()
            )  # Get the item price from the entry field

            gst_rate = float(
                gst_entries[i].get()
            )  # Get the item GST rate from the entry field
            qty = int(qty_entries[i].get())  # Get the item Qty from the entry field

            items.append(
                (item, price, gst_rate, qty)
            )  # Append item details to the list
```

```python
        # Generate unique bill number using current timestamp
        timestamp = datetime.now().strftime("%Y%m%d%H%M%S")
        bill_number = f"{timestamp}"

        # Create bill directory if it doesn't exist
        if not os.path.exists("savbills"):
            os.makedirs("savbills")

        # Create bill file path
        bill_file_path = f"savbills/{bill_number}.txt"

        # Create bill content
        bill_content = f"""                          Welcome to ThakurJi Shop

Customer Name: {customer_name}
Phone Number: {phone_number}
Bill Number: {bill_number}

==============================================================================
Item            Price           Qty             GST                     Total
=============================================================================="""

        # Loop to calculate total amount for each item and add to the bill content
        i = 0
        for item, price, gst_rate, qty in items:
            i += 1
            item_total = calculate_gst(
                price, gst_rate, qty
            )  # Calculate the total amount for the item
            total_amount += item_total  # Update the total amount

            # Add item details and total amount to the bill content
            bill_content += f"{i}. {item}         {price}              {qty}              {gst_rate}%           {item_total}\n\n"
            temp_i_detail = f"{i}. {item}         {price}              {qty}              {gst_rate}%           {item_total}\n\n"

        bill_content += "------------------------------------------------------------------------------\n"
        bill_content += f"TOTAL AMOUNT:\t\t\t\t{total_amount}\n"  # Add the final total amount of the bill to the bill content
        bill_content += "------------------------------------------------------------------------------\n"

        # Save the bill to a file
        with open(bill_file_path, "w") as file:
            file.write(bill_content)

        # Clear the output text widget
        output_text.delete(1.0, tk.END)

        # Display the bill content in the output text widget
        output_text.insert(tk.END, bill_content)

        # Display success message
        output_text.insert(tk.END, f"\nBill saved successfully as {bill_number}.txt")
```

```python
def welcome_bill():
    output_text.delete("1.0", END)
    output_text.insert(
        END,
        """                           Welcome to ThakurJi Shop

Customer Name:
Phone Number:
Bill Number:

===============================================================================
Item            Price           Qty             GST                          Total
===============================================================================""",
    )


# Function to search for a bill
def search_bill():
    search_bill_number = search_entry.get()  # Get the bill number to search

    # Clear the output text widget
    output_text.delete(1.0, tk.END)

    # Search for the bill file

    file_path = f"savbills\{search_bill_number}.txt"  # Create the full file path

    if os.path.exists(file_path):
        # Read and display the bill content
        with open(file_path, "r") as bill_file:
            bill_content = bill_file.read()
            output_text.insert(tk.END, bill_content)
        return
    else:
        print(f"The file {search_bill_number} does not exist in the folder.")

    # Display error message if bill is not found
    messagebox.showinfo("TryAgain", "Bill Not Found!!")
    output_text.insert(tk.END, "Bill not found.")
```

```python
# Create the main window
window = tk.Tk()
window.geometry("1920x1080+0+0")
window.title("Prashant Biling Software")
window.iconbitmap("favicon.ico")
title = tk.Label(
    window,
    text="Biling System",
    bd=5,
    relief=GROOVE,
    bg="#EDEADE",
    fg="black",
    font=("Baloo Bhai", 30, "bold"),
    pady=0,
).pack(fill=X)

# Create a frame for customer details
customer_frame = LabelFrame(
    window,
    text="Customer details:",
    bd=5,
    fg="green",
    font=("Times New Roman", 12, "bold"),
)
customer_frame.pack()

# Create labels and entry fields for customer details
customer_name_label = tk.Label(
    customer_frame,
    text="Customer Name:",
    bd=5,
    relief=GROOVE,
    bg="#1A8A70",
    fg="white",
    pady=10,
    font=("Times New Roman", 15, "bold"),
)
customer_name_label.pack(side="left")
customer_name_entry = tk.Entry(customer_frame, width=25, bd=5, relief=SUNKEN)
customer_name_entry.pack(side="left")

phone_number_label = tk.Label(
    customer_frame,
    text="Phone Number:",
    bd=5,
    relief=GROOVE,
    bg="#1A8A70",
    fg="white",
    pady=10,
    font=("Times New Roman", 15, "bold"),
)
phone_number_label.pack(side="left")
phone_number_entry = tk.Entry(customer_frame, width=25, bd=5, relief=SUNKEN)
phone_number_entry.pack(side="right")
```

तत त्वं पूषन् अपावृणु
केन्द्रीय विद्यालय संगठन

```
# Lists of Products details
item_entries = []
price_entries = []
gst_entries = []
qty_entries = []

i = 0


def add_item():
    global i
    i = i + 1

    # Create a frame for item details
    item_frame = LabelFrame(
        window,
        bd=8,
        text=f"   ITEM DETAILS:{i}   ",
        relief=GROOVE,
        font=("baloo bhai", 15, "bold"),
        fg="red",
    )
    item_frame.place(x=780, y=135 + 25)
    # Create labels and entry fields for item details
    item_label = tk.Label(
        item_frame, fg="black", text=f"ITEM NAME:", font=("Baloo bhai", 12, "bold")
    )
    item_label.pack()

    item_entry = tk.Entry(item_frame)
    item_entry.pack()
    item_entries.append(item_entry)

    price_label = tk.Label(
        item_frame, fg="black", text="Item Price:", font=("Baloo bhai", 12, "bold")
    )
    price_label.pack()

    price_entry = tk.Entry(item_frame)
    price_entry.pack()
    price_entries.append(price_entry)

    item_label = tk.Label(
        item_frame, fg="black", text="Quantity:", font=("Baloo bhai", 12, "bold")
    )
    item_label.pack()

    qty_entry = tk.Entry(item_frame)
    qty_entry.pack()
    qty_entries.append(qty_entry)

    gst_label = tk.Label(
        item_frame,
        fg="black",
        text="Item GST Rate (%):",
        font=("Baloo bhai", 12, "bold"),
    )
    gst_label.pack()
```

```python
    gst_entry = tk.Entry(item_frame)
    gst_entry.pack()
    gst_entries.append(gst_entry)

    if i != 1:
        if customer_name_entry.get() == "" or phone_number_entry.get() == "":
            messagebox.showerror("Don't Forget", "Customer details are must")
        elif (
            not (phone_number_entry.get().isnumeric())
            or len(phone_number_entry.get()) != 10
        ):
            messagebox.showerror("Invalid Details", "Enter a Valid Phone Number")


# Create an initial set of entry fields
add_item()


# Create a button to add more items
add_item_button = tk.Button(window, text="Add Item", bd=5, fg="green", command=add_item)
add_item_button.place(x=865 - 20, y=385 + 25)


# Create a button to generate the bill
generate_button = tk.Button(
    window,
    text="GenerateBill",
    bg="cadetblue",
    fg="white",
    bd=4,
    pady=0,
    width=12,
    font=("baloo bhai", 10, "bold"),
    command=generate_bill,
)
generate_button.place(x=845 - 20, y=425 + 25)

# Create a label and entry field for searching bill
search_label = tk.Label(
    window, text="Search Bill", fg="black", font=("baloo bhai", 13, "bold")
)
search_label.place(x=1136, y=200)

search_entry = tk.Entry(window, width=25, bd=5, relief=SUNKEN)
search_entry.place(x=1110, y=225)

# Create a button to search for a bill
search_button = tk.Button(
    window, text="Search", command=search_bill, width=5, bd=3, font=("arial 15 bold")
)
search_button.place(x=1115 + 36, y=255)
```

```
# ====Bill Area======
F5 = Frame(window, bd=10, relief=GROOVE)
F5.place(x=648, y=500, width=665, height=420)
bill_title = Label(
    F5, text="Bill Area", font=("baloo bhai", 16, "bold"), bd=7, relief=GROOVE
).pack(fill=X)
scrol_y = Scrollbar(F5, orient=VERTICAL)
output_text = Text(F5, yscrollcommand=scrol_y.set)
scrol_y.pack(side=RIGHT, fill=Y)
scrol_y.config(command=output_text.yview)
output_text.pack(fill=BOTH, expand=1)
welcome_bill()


# Start the main loop
window.mainloop()
```

CODE EXPLAINATION

- The given code is a Python program that implements a simple GST (Goods and Services Tax) Billing System using the tkinter library for GUI. The system allows users to enter customer details, add multiple items along with their prices, quantities, and GST rates, and then generate a bill with the total amount including GST.

❑ Here's a breakdown of the code:

➢ The code begins by importing necessary modules:

- tkinter for creating the graphical user interface.
- os for file system operations.
- datetime for generating a unique bill number based on the current timestamp.

➢ There are several functions defined in the code:

- **calculate_gst**: This function calculates the GST amount and total amount for a given item based on its price, GST rate, and quantity.

- **generate_bill**: This function is called when the user clicks the "GenerateBill" button. It performs various validations on customer details, retrieves item details, calculates the total amount, generates a unique bill number, creates a bill file, and saves the bill details to the file.

- **welcome_bill**: This function is used to clear the output_text widget and display a welcome message with a predefined format.

- **search_bill**: This function is called when the user clicks the "Search" button. It searches for a bill file based on the user-provided bill number and displays its content in the output_text widget.

- The main part of the code starts by creating the main tkinter window.
  - It sets the window size, title, and icon for the application.
  - It creates a title label with the text "Billing System" at the top of the window.

- A frame (**customer_frame**) is created to store customer details:
  - Labels and entry fields are added to enter customer name and phone number.

- Item details are stored in a list of entry fields:
  - Initially, one set of entry fields is created for item name, price, quantity, and GST rate.
  - The **add_item** function allows adding more sets of item entry fields when the "Add Item" button is clicked.
  - The user must enter customer details before adding items, and phone number validation is performed to ensure it contains only numbers and is 10 digits long.

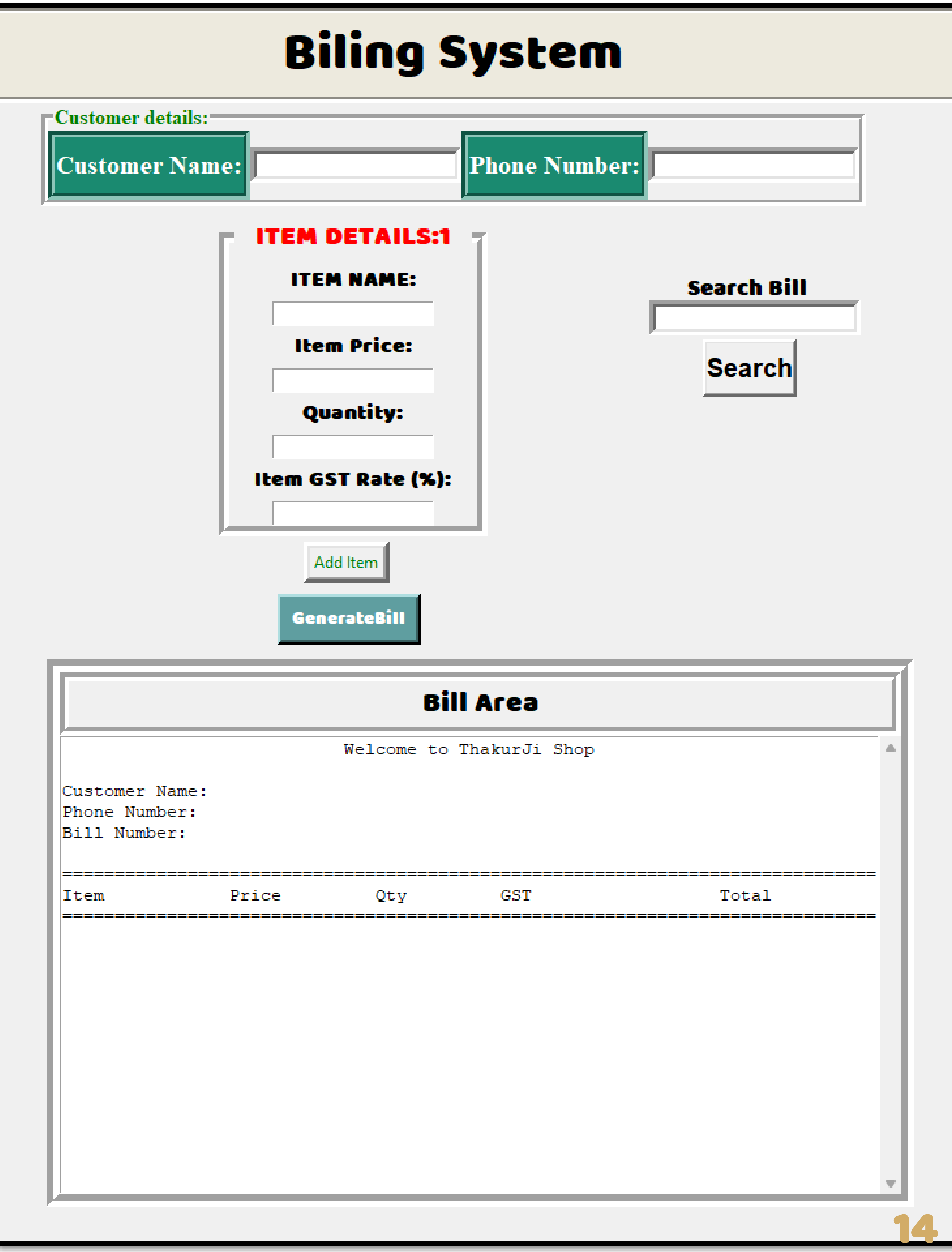
- Buttons for adding items and generating bills are created:
  - "Add Item" button calls the **add_item** function to add more sets of item entry fields.
  - "GenerateBill" button calls the **generate_bill** function to create the bill and save it to a file.

- A search section is provided to search for a specific bill:
  - A label and an entry field are created for entering the bill number.
  - The "Search" button calls the **search_bill** function to find and display the bill content.

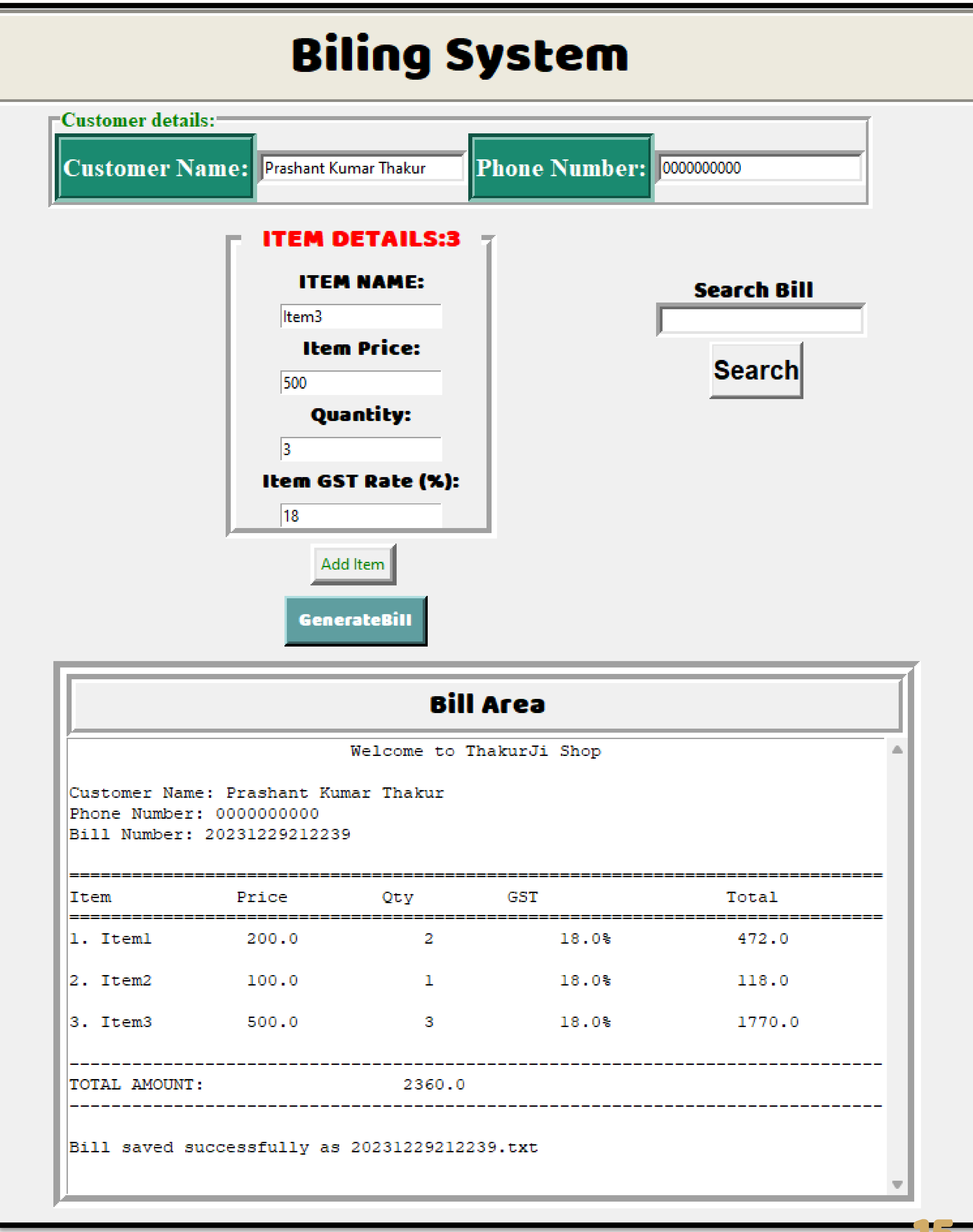
- An output_text widget is provided to display the generated bill:
  - The widget is placed inside a frame to manage its size and position.
  - The **welcome_bill** function sets a welcome message in the output_text widget.

- The main loop (**window.mainloop()**) runs the application and handles user interactions.

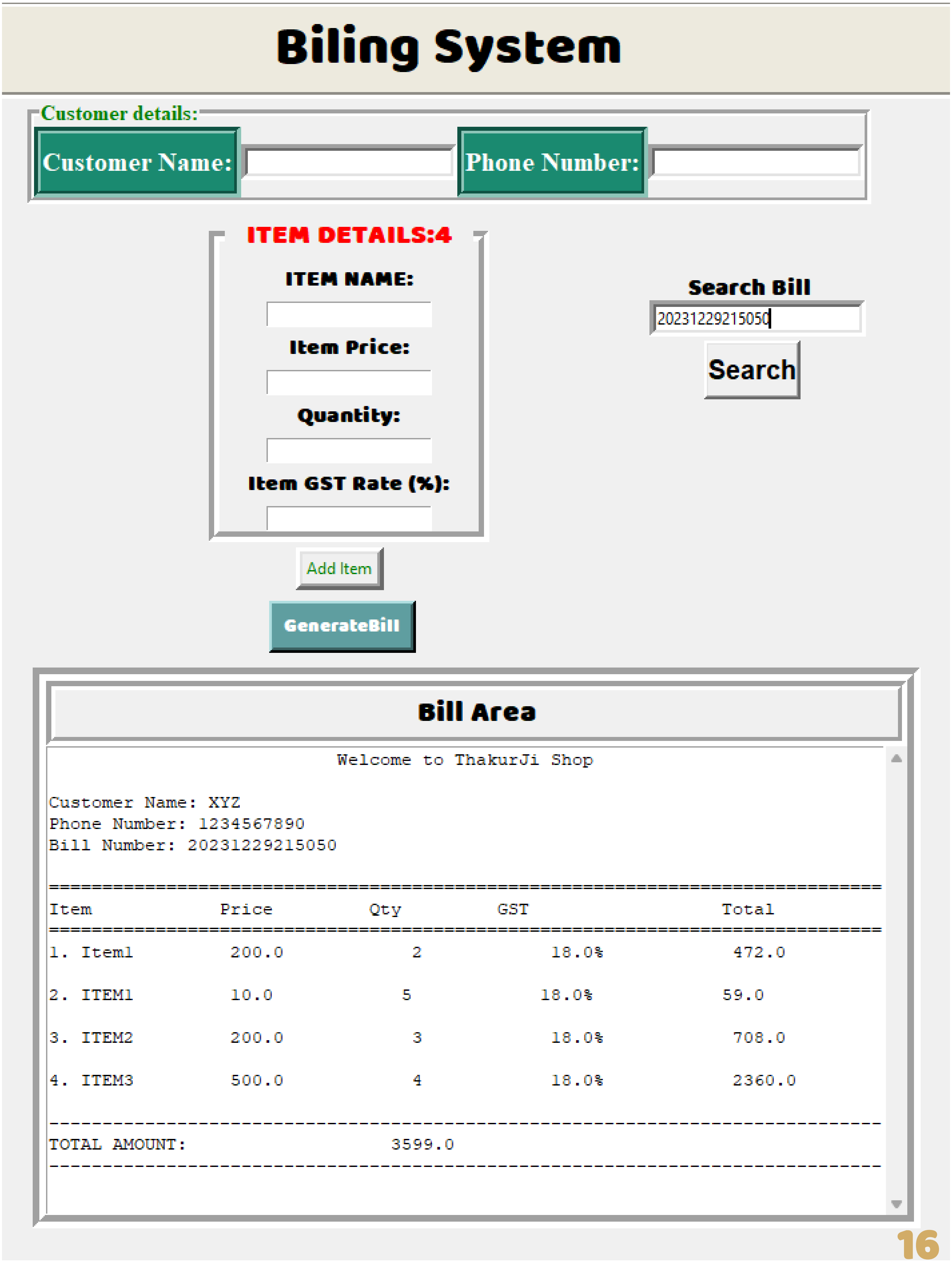
CODE OUTPUT


Biling System
Customer details:
Customer Name:
Phone Number:
ITEM DETAILS:1
ITEM NAME:
Item Price:
Quantity:
Item GST Rate (%):
Search Bill
Search
Add Item
GenerateBill
Bill Area
Welcome to ThakurJi Shop
Customer Name:
Phone Number:
Bill Number:
==
Item Price Qty GST Total
==

After Entering customer and item details Clicking **GenerateBill** Button

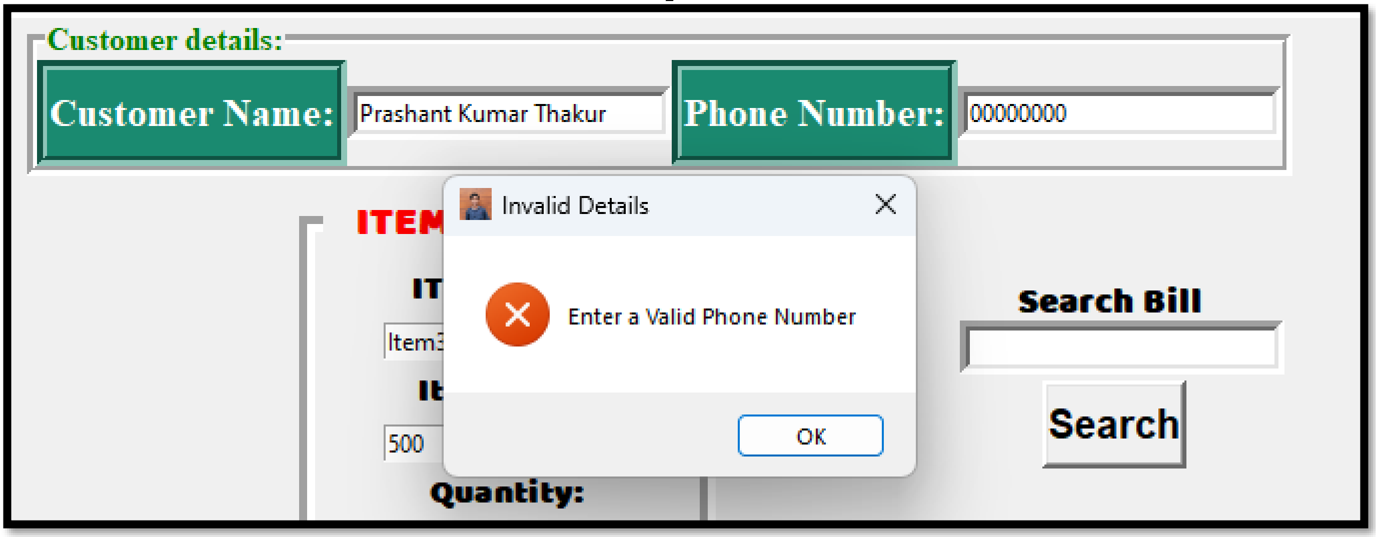


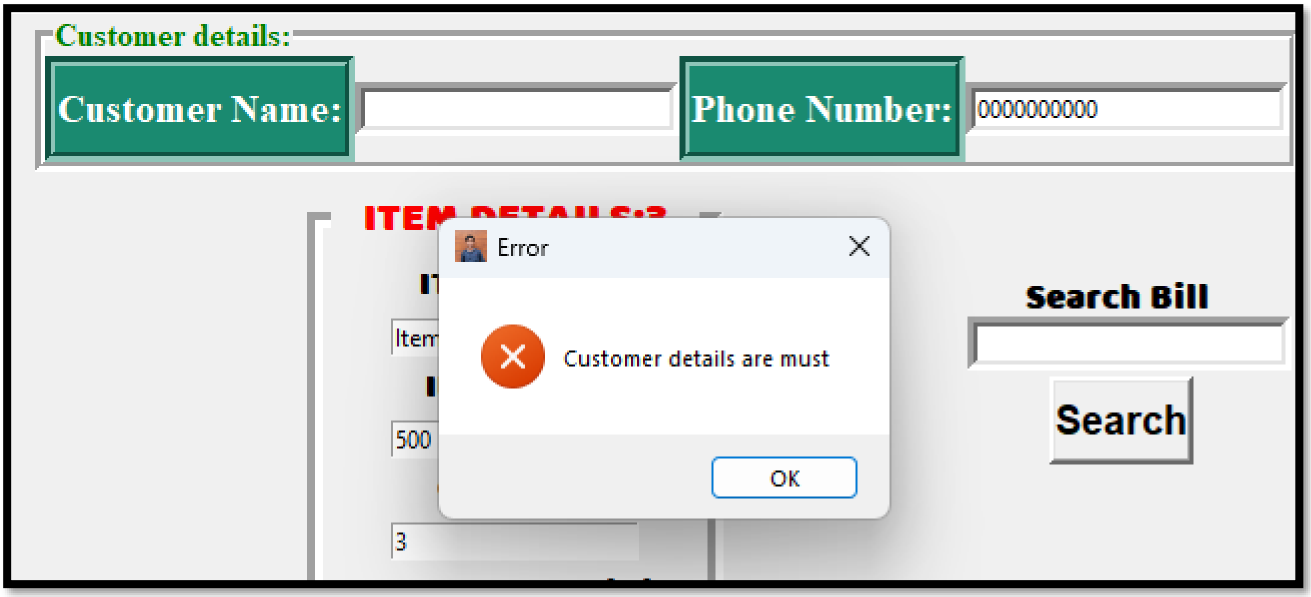
Searching a Bill with Bill Number

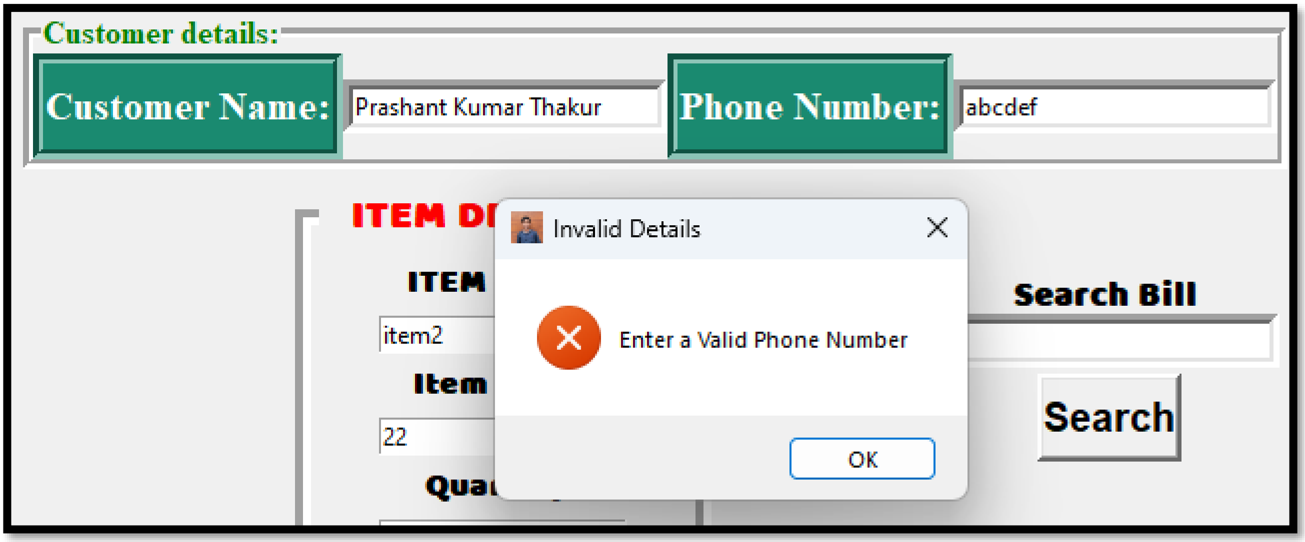


The Software also Handles Some User input related Exceptions

Below they are shown







STUDENT REFLECTION

Working on the Goods and Services Tax (GST) Billing System was a great learning experience for me. I got hands-on practice with Python programming and Tkinter GUI development, which helped me understand how to make programs work and take in user inputs. Figuring out and fixing issues during the project improved my problem-solving skills, making me better at finding and solving problems in my code. Even though I mainly worked on my own, the project showed me how real-world software development can be a team effort where people help each other out.

Beyond just coding, the project also showed me how businesses work behind the scenes. It helped me see that good software is closely connected to real-world challenges. Looking ahead, completing the GST Billing System project makes me interested in doing more. I'm thinking about adding things like saving data in a database, making security better, and getting real-time updates from other programs. Overall, this project didn't just make me better at coding but also helped me see how technology and real businesses go hand in hand.

BIBLIOGRAPHY

- Sumita Arora- Computer Science with Python-Class XII
- python.org
- https://docs.python.org/3/library/tk.html
- https://youtube.com/playlist?list=PLu0W_9lII9ajLcqRcj4PoEihkukF_OTzA&si=02ZECFXoJpmfO0NP
- https://www.google.com
- Researches From Internet